

R4Tun: LLM-driven adaptive segmental tunnel lining segmentation in point clouds

Authors: Xinghui Tao, Guangming Wang *, Jelena Ninić, Brian Sheil

Affiliations: a: Construction Engineering, University of Cambridge, Trumpington Street, Cambridge, CB2 1PZ, Cambridge, UK b: Department of Engineering, Durham University, Stockton Road, Durham, DH1 3LE, Durham, UK

Corresponding author: Guangming Wang, gw462@cam.ac.uk, Construction Engineering, University of Cambridge, Trumpington Street, Cambridge, CB2 1PZ, Cambridge, UK

Abstract

Automated inspection of segmental tunnel linings requires reliable segmentation of structural components from 3D point clouds, yet current pipelines depend on expert-tuned parameters that degrade when tunnel conditions vary. This paper presents R4Tun, a large language model (LLM) driven adaptation framework that augments an expert-designed pipeline (SAM4Tun) with bounded, context-aware parameter tuning. R4Tun provides each stage agent with structured context comprising memory (reference characteristics and parameters), state (intermediate pipeline outputs), and knowledge (domain-specific tuning rules), from which the agent produces bounded parameter updates via chain-of-thought reasoning. The framework was evaluated on 30 Seg2Tunnel subsets spanning 13 regular and 17 complex tunnels using a cumulative ablation of context components across three LLMs (Claude Opus 4.6, GPT-5.4, Gemini 3 Flash). The full design improved mean mIoU by 108.7–118.7% overall (from 0.150 to 0.313–0.328), by 70.1–83.8% for regular tunnels (from 0.291 to 0.495–0.535), and by 302.4–321.4% for complex tunnels (from 0.042 to 0.169–0.177), with state contributing the largest incremental gain (paired Cohen’s $d = 1.32$ – 1.61 , $p < 0.0001$). Cross-model analysis of 1,350 adapted parameter files revealed that all three LLMs independently converge on the same critical parameters and tunnel-family clustering, indicating adaptations are driven by tunnel characteristics rather than LLM-specific biases.

Keywords: Segmental tunnel lining; Point cloud segmentation; Tunnel inspection; Large language models; Parameter adaptation; Multi-agent systems

Highlights

- Expert-tuned tunnel segmentation degrades sharply when tunnel conditions depart from the reference (mIoU drops from 0.291 on regular to 0.042 on complex tunnels).
- R4Tun augments a fixed pipeline with LLM-driven adaptation using structured context: memory, state, and domain knowledge.
- Validated on 30 subsets covering two diameters, two ring lengths, two segment counts, and two joint types across three independent LLMs.
- Mean mIoU improved by more than 100% overall with bootstrap 95% CIs excluding zero for all three LLMs; per-class IoU improved broadly across all structural classes.

- Analysis of 1,350 adapted parameter files shows three LLMs converge on the same 18 critical parameters and tunnel-family adaptation patterns.
-

1. Introduction

1.1 Context and motivation

Automated inspection of segmental tunnel linings is increasingly required to support structural health assessment and long-term operational safety, given the scale, frequency, and access constraints of modern tunnel networks [1]. Reliable segmentation of structural components from 3D point-cloud data is a prerequisite for downstream deformation analysis and defect localisation [2]. Although modern laser scanning enables high-fidelity tunnel capture, the resulting measurements contain mixed structural elements, occlusions, and noise, making direct extraction of lining components challenging [3,4].

Three main approaches exist for tunnel point-cloud segmentation. Feature-engineering encodes domain knowledge into deterministic geometric rules that are interpretable but brittle under changed conditions [5,6]. Supervised deep learning reduces hand-crafting but requires large labelled datasets, substantial computational resources, and periodic retraining [7–10]. Foundation-model pipelines such as SAM4Tun [11] combine geometric preprocessing with SAM-based prompting [12] to achieve strong performance under expert tuning, yet remain highly sensitive to approximately 60 pipeline parameters. When tunnel geometry or scanning conditions deviate from the tuning reference, segmentation quality degrades sharply. None of these approaches combines adaptability to new conditions with interpretability of the adaptation process.

1.2 Problem statement

In practice, a fixed expert-tuned configuration achieves mean mIoU of only 0.042 on complex tunnels compared with 0.291 on regular tunnels (including continuous) for which it was calibrated. This instability causes repeated expert retuning, additional quality control, and lower confidence in automation for field deployment across diverse tunnel networks.

1.3 Research gap

Existing studies have shown that both supervised deep learning and foundation-model pipelines can achieve strong segmentation performance under controlled conditions. However, most evaluations assess accuracy on a fixed test set and do not demonstrate whether performance remains reliable when tunnel geometry, diameter, ring structure, or scanning conditions change. Feature-engineered pipelines are interpretable but lack adaptability; deep-learning models adapt through training data but lack interpretability; foundation-model pipelines reduce annotation demand but remain sensitive to parameter configuration. No existing approach integrates LLM reasoning with expert-designed pipelines to provide adaptive, interpretable parameter tuning for infrastructure applications.

1.4 Objectives

This paper develops R4Tun, an LLM-driven adaptation framework that augments an expert-designed tunnel segmentation pipeline so that stage parameters can be adjusted to changing tunnel conditions without expert tuning. The study addresses four specific aims:

1. **Design:** Develop a framework in which LLM agents adapt the parameters of a fixed pipeline using structured context from memory, state, and domain knowledge, without modifying the pipeline’s algorithms.
2. **Component isolation:** Quantify the incremental contribution of each context component through cumulative ablation across 30 tunnels and three LLMs.
3. **Cross-model validation:** Assess whether adaptation behaviour is consistent across three independent LLMs with different architectures and training data.
4. **Sensitivity analysis:** Identify which parameters are adapted, whether adaptations are tunnel-responsive or baseline corrections, and whether they are driven by objective tunnel characteristics.

1.5 Paper organisation

Section 2 reviews related work. Section 3 presents materials and methods, including the R4Tun architecture, dataset, experimental design, and evaluation. Section 4 reports results. Section 5 discusses findings, implications, and limitations. Section 6 concludes.

2. Related work

2.1 Tunnel point-cloud segmentation

Feature-engineered approaches encode domain knowledge into deterministic geometric rules — thresholds on curvature, radius, line features, or clustering criteria [5,6,13,14]. Because these rules are explicit and auditable, they remain widely used in safety-critical inspection. However, they require extensive manual reconfiguration when conditions change [2]. Supervised deep-learning methods reduce reliance on hand-crafted rules by learning hierarchical representations from annotated point clouds [7–10,15,16]. These models require large labelled datasets, significant computational resources, and periodic retraining when deployed across new tunnel typologies; their internal decision logic also remains opaque.

2.2 Foundation models and prompt-based segmentation

Foundation models pre-trained on large datasets enable zero-shot segmentation guided by prompts [12]. SAM has been applied to infrastructure tasks including crack detection and Scan-to-BIM workflows [17–19]. For tunnel linings, SAM4Tun [11] combines geometric preprocessing — unfolding to a 2D depth map, denoising, enhancement — with Hough-transform-based joint detection and template-based SAM prompting. Performance depends critically on pipeline parameters that control preprocessing, prompt generation, and post-processing. When tunnel geometry or point-cloud quality deviates from the tuning reference, these parameters become misspecified and segmentation quality degrades.

2.3 LLMs as reasoning agents in engineering tasks

Reasoning-enabled LLMs incorporate step-by-step reasoning traces, improving logical consistency and verifiability [20,21]. Multi-agent architectures assign specialised roles coordinated through shared context [22–24]. Context engineering — the systematic design of information provided to reasoning models — strongly influences performance [25–27]. In civil engineering, LLMs have been applied as assistants for design, analysis, and code generation [28,29], but integrating reasoning models to enhance the adaptability of expert-designed pipelines while preserving interpretability and overridability remains an open challenge.

2.4 Synthesis

No existing approach combines adaptability with interpretability by allowing an expert-designed pipeline to adjust its parameters systematically to new tunnel conditions while keeping every decision traceable and overridable. This gap motivates the present study.

3. Materials and methods

3.1 Problem definition

The task is semantic segmentation of segmental tunnel linings from terrestrial laser scanning (TLS) point clouds. Given a raw point cloud of a tunnel section, the goal is to assign each point a structural label: background, key block (K), base blocks (B1, B2), and adjacent blocks (A1–A3), with an additional A4 class for 7-segment complex tunnels.

3.2 Dataset

The Seg2Tunnel benchmark comprises 30 subsets from five real tunnels scanned with a Leica C10 scanner (Table 1). The subsets span systematic variation along four structural dimensions, providing a stratified evaluation design.

Table 1: Dataset properties

Property	Regular (T1, T2)	Continuous (T3)	Complex (T4, T5)
Inner diameter	5.5 m	5.5 m	7.5 m
Ring length	1.2 m	1.2 m	1.8 m
Segments per ring	6	6	7
Joint type	Staggered	Continuous	Complex interleaved
Scanning	Single-station (Wuxi)	Multi-station	Single-station, offset centre (Fuzhou)
Evaluation schema	6-class	6-class	7-class
Count	10 subsets	3 subsets	17 subsets

Regular and continuous tunnels are grouped as “regular” ($n = 13$), sharing the 5.5 m diameter and 6-class schema. Complex tunnels ($n = 17$) differ from the reference in diameter (+36%), ring length (+50%),

segment count (+1), and scanning geometry. The baseline was expert-tuned on reference tunnel T2-2 (diameter 5.60 m, density 2,466 pts/m³, mIoU 0.531).

3.3 Proposed method

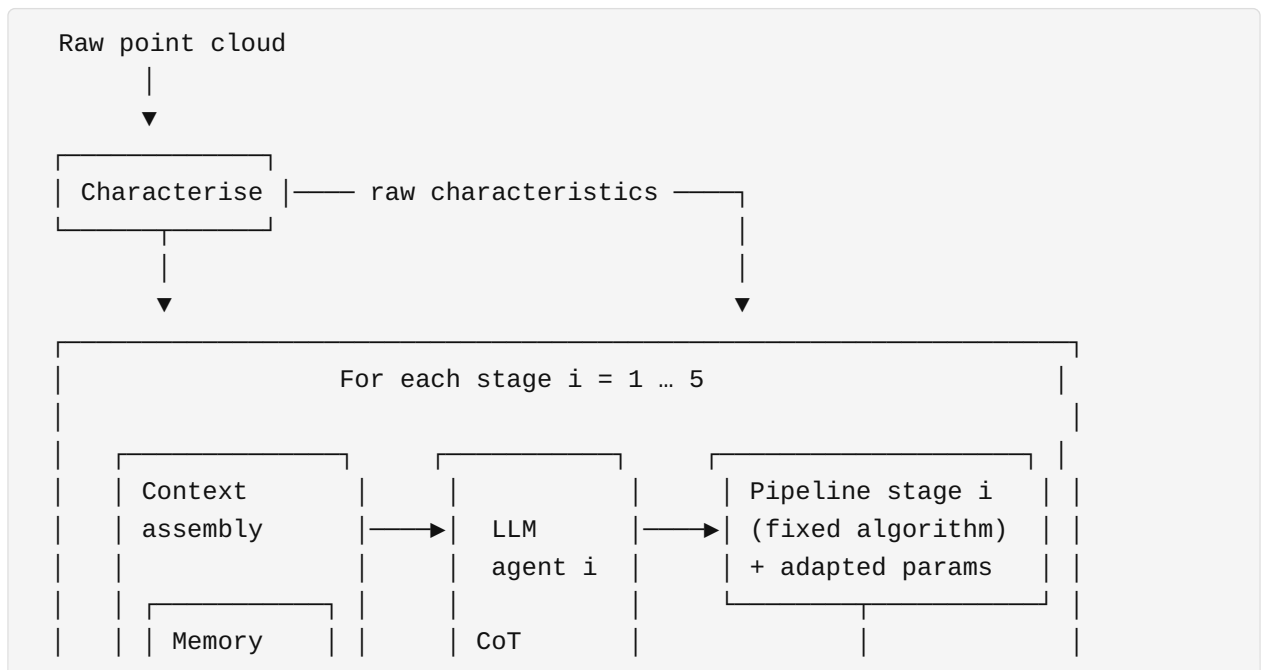
3.3.1 Overview and workflow

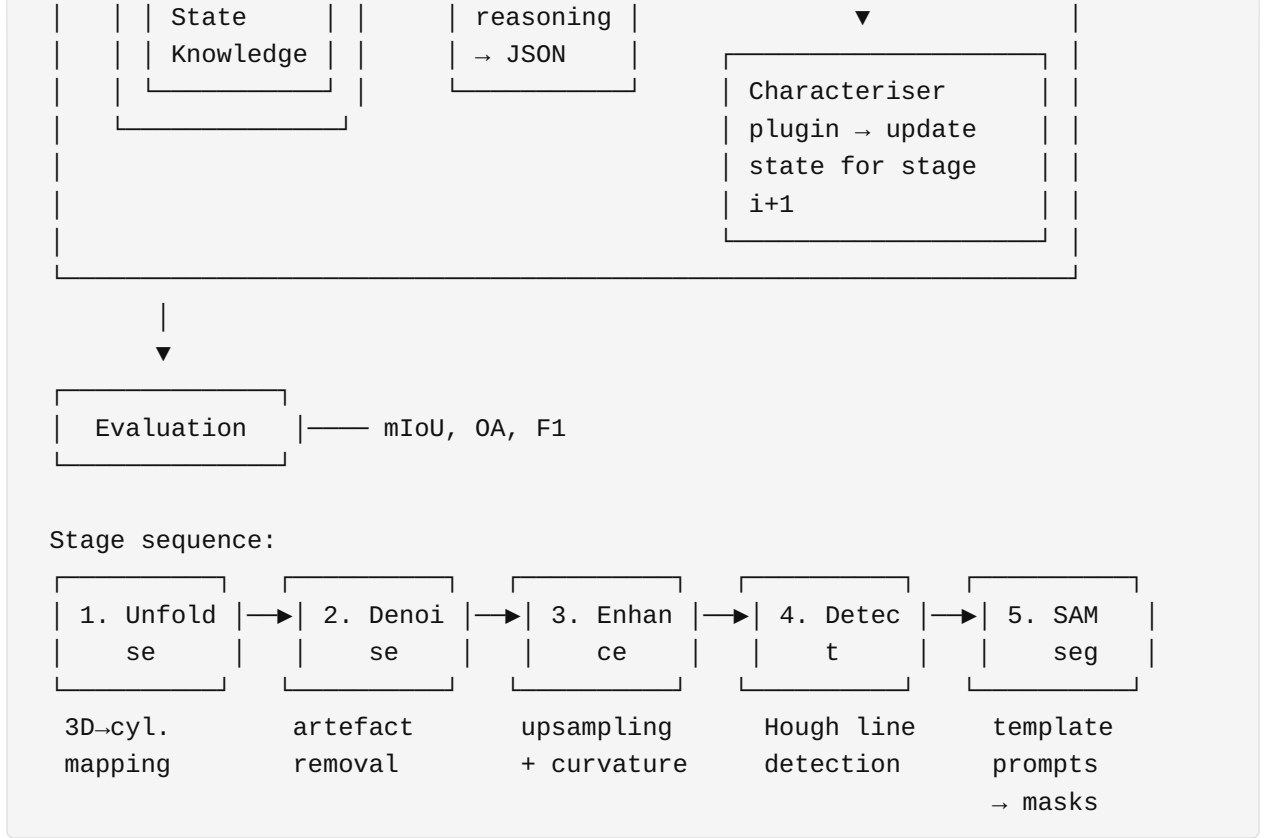
R4Tun augments the five-stage SAM4Tun pipeline with an LLM-driven adaptation layer. The pipeline’s sequential stages — Unfolding, Denoising, Enhancing, Detecting, and SAM segmentation — remain fixed; only the parameter JSON files fed to each stage change. The workflow for each tunnel proceeds as follows:

1. **Characterisation:** Extract raw tunnel characteristics (geometry, density, coordinate ranges) from the input point cloud.
2. **For each stage (sequentially):**
 3. (a) The orchestrator assembles a structured prompt from the current context level (memory, state, and/or knowledge).
 4. (b) The LLM receives the prompt and returns a JSON parameter file via chain-of-thought reasoning.
 5. (c) The pipeline stage executes with the adapted parameters.
 6. (d) A characteriser plugin extracts structured statistics from the stage output, updating the state for subsequent stages.
7. **Evaluation:** Compute per-point semantic labels and mIoU against ground truth.

Each stage’s parameters are inferred independently from the reference baseline — the LLM does not see its own prior-stage parameter outputs, preventing self-reinforcement of errors. The five stage scripts and evaluation code are identical across all ablation conditions; only the parameter JSONs change. Fig. 1 illustrates the end-to-end workflow.

Fig. 1: R4Tun system architecture and workflow.





Caption: End-to-end R4Tun workflow. The five SAM4Tun pipeline stages (bottom row) remain algorithmically fixed. For each stage, the LLM agent receives assembled context (memory, state, knowledge), reasons via CoT, and outputs an adapted parameter JSON. A characteriser plugin updates the state after each stage. Only parameter JSONs change between ablation conditions.

3.3.2 Pipeline stages

Stage 1 — Unfolding. Maps the 3D point cloud to cylindrical coordinates (r, θ, h) via RANSAC-based centreline fitting. Key parameters: slice half-thickness, slice spacing factor, vertical filter window, diameter.

Stage 2 — Denoising. Removes non-structural artefacts using grid-based radial-density filtering with radial masks. Key parameters: mask_r_low, mask_r_high, y_step, z_step, gradient threshold, smoothing settings, default radial cutoff.

Stage 3 — Enhancing. Improves geometric continuity through three-stage progressive upsampling and curvature-guided point insertion. Key parameters: upsampling target distances, curvature threshold, depth thresholds, interpolation radius.

Stage 4 — Detecting. Applies Hough-transform line detection to extract ring boundaries from the depth map. Key parameters: binary threshold, Hough thresholds (oblique, horizontal, vertical), line settings, merge distance, ring spacing constant.

Stage 5 — SAM segmentation. Constructs template-based point and mask prompts from detected boundaries, applies SAM (ViT-H) to produce 2D segment masks, and reprojects labels into 3D. Key

parameters: segment_per_ring, segment_order, segment dimensions, processing settings, prompt point templates.

3.3.3 Context components

Each stage agent receives structured context comprising three components, each defined in an implementable way:

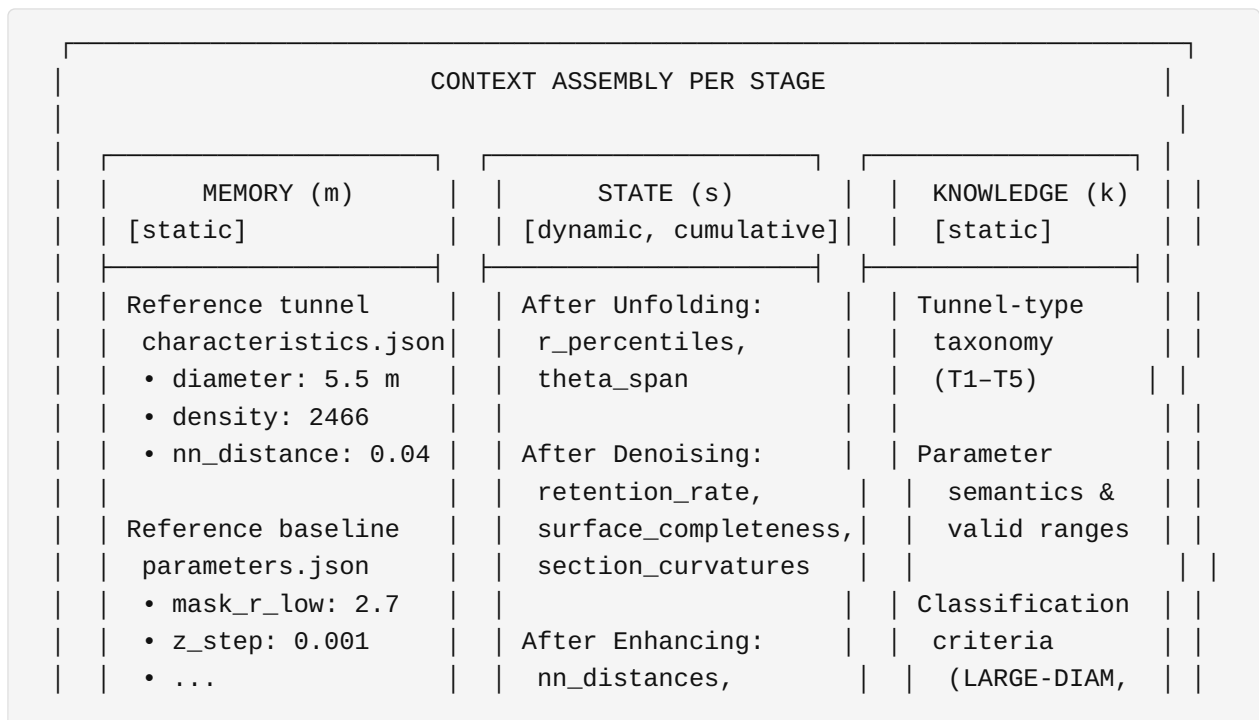
Memory stores the reference tunnel’s characteristics (a JSON file containing geometry, point density, coordinate ranges, nearest-neighbour distances) alongside the expert-tuned reference parameters (the baseline JSON for that stage). The agent compares the current tunnel’s characteristics against this reference to quantify deviation and anchor its parameter adjustments. Memory is static: it does not change between stages.

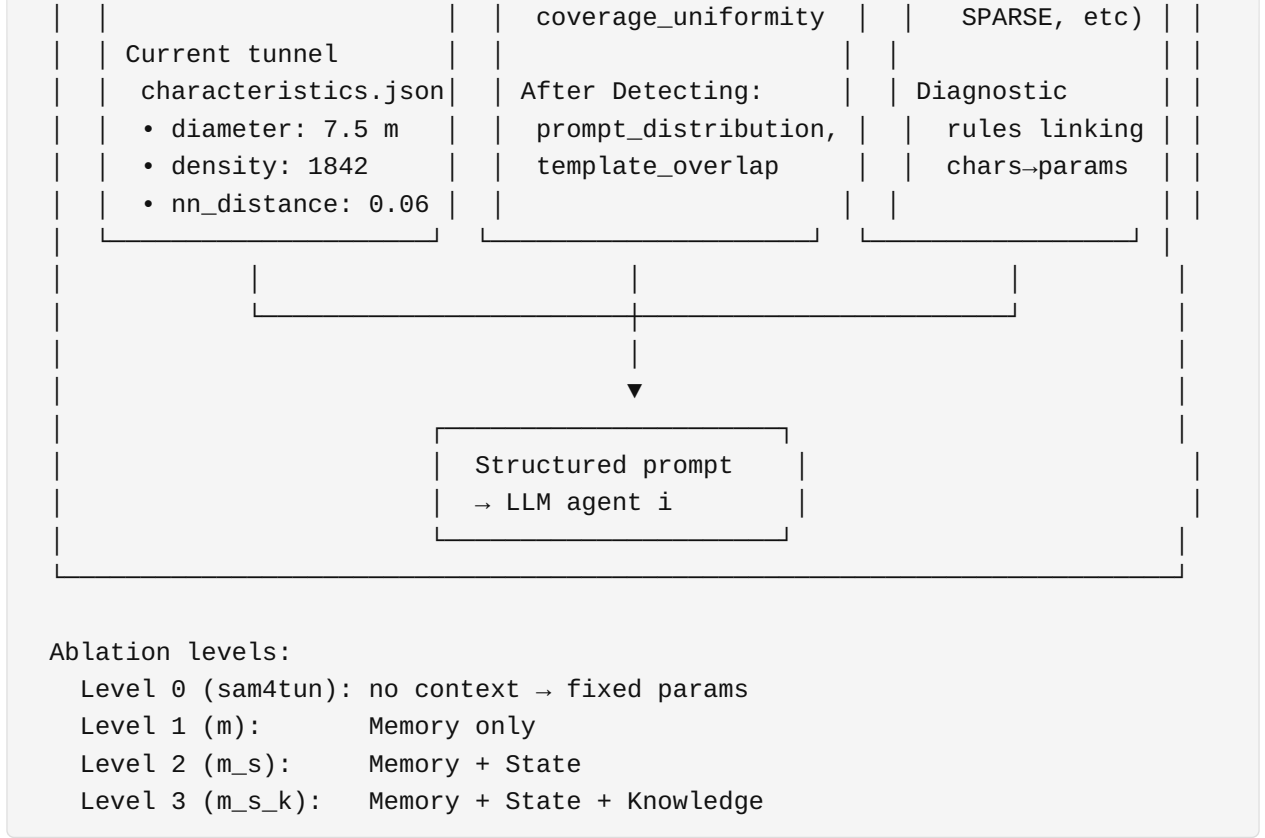
State captures cumulative geometric and statistical properties after each pipeline stage executes. After each stage, a characteriser plugin extracts a structured JSON summary: radial percentiles (p10, p99) and theta span after unfolding; retention rate, surface completeness, and section curvatures after denoising; nearest-neighbour distances and coverage uniformity after enhancing; prompt distribution and template overlap after detecting. These cumulative summaries are injected into subsequent agents’ prompts. State is dynamic: it grows as stages execute.

Knowledge supplies domain-specific guidance in human-readable markdown documents. Each stage has its own knowledge document covering: tunnel-type taxonomy (T1–T5 variations); parameter semantics and empirically validated ranges; classification criteria for tunnel conditions; and diagnostic rules linking characteristics to parameter adjustments. Knowledge is authored once and shared across all tunnels.

The three context components and their data flow are illustrated in Fig. 2.

Fig. 2: Context design — Memory, State, and Knowledge.





Caption: The three context components provided to each stage agent. Memory anchors the agent to the reference tunnel. State grows cumulatively as each pipeline stage executes, providing intermediate feedback. Knowledge supplies domain rules and taxonomy. The ablation levels incrementally add each component.

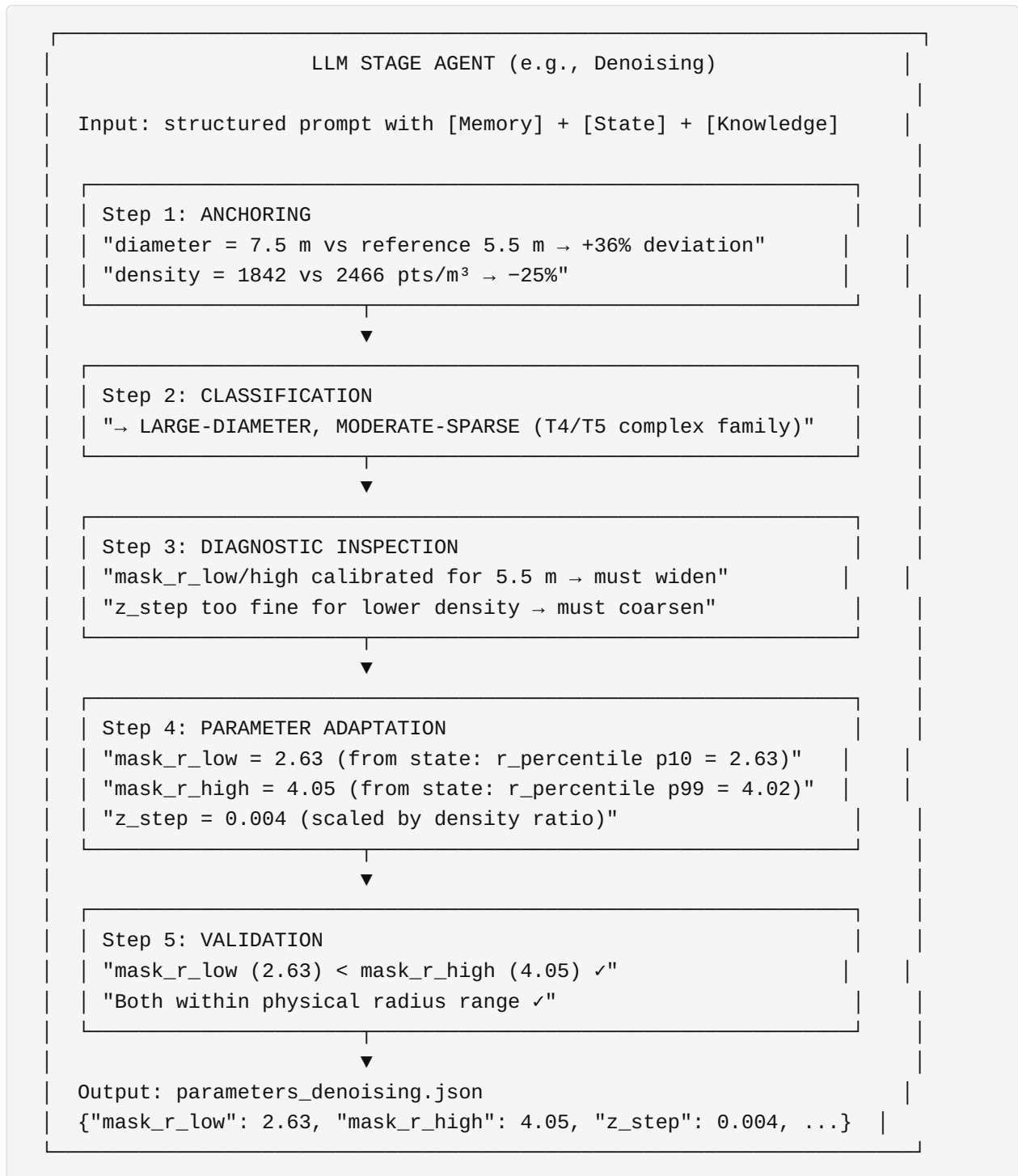
3.3.4 Chain-of-thought reasoning

The agent’s reasoning follows a structured five-step chain-of-thought (CoT) protocol:

1. **Anchoring:** Quantify how the current tunnel deviates from the reference (e.g., “diameter is 7.5 m vs 5.5 m reference, +36%”).
2. **Classification:** Categorise the tunnel condition (e.g., LARGE-DIAMETER, SPARSE, SIMILAR).
3. **Diagnostic inspection:** Identify which parameters are implicated by the deviation (e.g., “mask_r_low must increase to accommodate larger radius”).
4. **Parameter adaptation:** Propose bounded updates with evidence (e.g., “set mask_r_low = 2.65 based on r_percentile p10 = 2.63”).
5. **Validation:** Check logical consistency (e.g., “mask_r_low < mask_r_high, both within tunnel radius range”).

The agent outputs a single JSON object matching the reference schema exactly. Fig. 3 illustrates the CoT flow for a single stage agent.

Fig. 3: Chain-of-thought reasoning protocol for a single stage agent.



Caption: Five-step CoT reasoning protocol. The agent grounds each parameter change in quantitative evidence from state and anchors deviations against memory. Knowledge guides classification and diagnostic rules. The output is a schema-conformant JSON.

3.3.5 LLM configuration

All three LLMs were accessed via their respective commercial APIs with the following settings:

Setting	Value
Models	Claude Opus 4.6 (Anthropic), GPT-5.4 (OpenAI), Gemini 3 Flash (Google)
Max tokens	16,384
Temperature	Default (API default, not overridden)
Timeout	300 seconds per call
Prompt format	Markdown with JSON code fences
Failure handling	If JSON extraction fails, raw response is logged; stage raises an error

All three LLMs received identical prompts and context for each condition. No model-specific prompt tuning was performed.

3.4 Baseline

The baseline (“sam4tun”) applies a single set of expert-tuned parameters to all 30 tunnels without any per-tunnel adaptation. This represents the deterministic execution of the expert’s tuning rules: the rules encoded in SAM4Tun’s code produce one fixed configuration, and that configuration is applied uniformly. The baseline therefore serves as the rule-only reference against which LLM-driven adaptation is measured.

3.5 Experimental design

3.5.1 Ablation ladder

The experiment follows a cumulative ablation design with four conditions, each adding one context component:

Table 2: Ablation conditions

Level	Code	Condition	What the LLM sees
0	sam4tun	Baseline (fixed params)	Nothing — fixed default parameters for all tunnels
1	m	Memory	Reference tunnel characteristics and reference parameters
2	m_s	Memory + State	+ intermediate pipeline stage outputs (cumulative characteristics)
3	m_s_k	Memory + State + Knowledge	+ domain knowledge (parameter semantics, adaptation rules, taxonomy)

This cumulative design isolates the incremental contribution of each component.

3.5.2 Cross-LLM validation

Each ablation condition was run with three independent LLMs under identical prompts to assess whether adaptation behaviour depends on the specific model. The three LLMs differ in architecture, training data, and vendor, providing independent replication.

3.5.3 Statistical testing

For each condition and LLM, paired differences $\Delta_i = \text{mIoU_condition_i} - \text{mIoU_baseline_i}$ were computed for each tunnel i . The following statistical procedures were applied:

- **Paired t-tests** (two-sided, $\alpha = 0.05$) per tunnel family and overall.
- **Wilcoxon signed-rank tests** as a non-parametric sensitivity check (paired differences need not be normal at $n = 13$ or $n = 17$).
- **Tunnel-level bootstrap 95% CIs** (10,000 resamples of 30 tunnels with replacement) on mean paired ΔmIoU . These quantify uncertainty over benchmark composition, not SAM or LLM rerun noise.
- **One-sided binomial sign tests** for the knowledge increment (count of tunnels with strictly positive Δ vs fair coin).
- **Paired Cohen’s d** as effect-size measure for each transition.

3.6 Evaluation metrics

Segmentation quality is measured by mean Intersection-over-Union (mIoU): $\text{IoU_c} = \text{TP_c} / (\text{TP_c} + \text{FP_c} + \text{FN_c})$, $\text{mIoU} = (1/C) \sum \text{IoU_c}$, where $C = 6$ (regular) or 7 (complex). Overall accuracy (OA) and macro F1 are also computed; mIoU is the primary metric.

3.7 Sensitivity analysis

Three complementary analyses were conducted on 1,350 adapted parameter files (30 tunnels $\times$ 3 LLMs $\times$ 3 ablation conditions $\times$ 5 stages):

1. **Critical parameter identification:** For each (stage, parameter) pair: trigger frequency, cross-LLM agreement, coefficient of variation (CV). Parameters with $\text{CV} \geq 0.06$ and tunnel-dependent values are classified as tunnel-responsive; those with $\text{CV} \approx 0$ as baseline corrections.
2. **Cross-LLM consistency:** Per-LLM tunnel counts, value ranges, and tunnel-family clustering for all always-trigger parameters.
3. **Characteristic-to-parameter correlation:** Spearman rank correlations ($N = 30$) between every characteristic field and every adapted parameter value, cross-validated against CoT text-mining.

4. Results

4.1 Main quantitative results

Table 3: Mean mIoU by condition and LLM (n = 30)

Condition	Opus 4.6	GPT-5.4	Gemini 3 Flash
sam4tun (baseline)	0.150	0.150	0.150
memory	0.144	0.182	0.199
memory+state	0.312	0.299	0.302
memory+state+knowledge	0.328	0.321	0.313

Table 4: Paired Δ mIoU vs baseline — overall (n = 30)

Condition	Statistic	Opus 4.6	GPT-5.4	Gemini 3 Flash
memory	Mean Δ	−0.006	+0.032	+0.049
	p-value	0.558	0.028	0.056
	Cohen’s d	−0.11	0.42	0.36
	Bootstrap 95% CI	[−0.027, 0.014]	[0.005, 0.058]	[0.006, 0.101]
memory+state	Mean Δ	+0.162	+0.149	+0.152
	p-value	< 0.0001	< 0.0001	< 0.0001
	Cohen’s d	1.61	1.32	1.46
	Bootstrap 95% CI	[0.126, 0.198]	[0.110, 0.190]	[0.117, 0.189]
m+s+k	Mean Δ	+0.178	+0.171	+0.163
	p-value	< 0.0001	< 0.0001	< 0.0001
	Cohen’s d	1.61	1.94	1.44
	Bootstrap 95% CI	[0.139, 0.216]	[0.140, 0.203]	[0.122, 0.203]

All bootstrap CIs for memory+state and m+s+k exclude zero across all three LLMs. Wilcoxon tests agree with t-test conclusions on overall and complex subsets (both $p < 0.0001$ for all LLMs).

Table 5: Mean mIoU by tunnel family

Family	n	sam4tun	memory	memory+state	m+s+k
Regular (all)	13	0.291	0.260–0.344	0.516–0.531	0.495–0.535
Complex	17	0.042	0.055–0.101	0.126–0.155	0.169–0.177
Overall	30	0.150	0.144–0.199	0.299–0.312	0.313–0.328

The fixed baseline degrades severely on complex tunnels (mIoU 0.042), where every pipeline assumption is outside its design envelope. The full R4Tun design lifts all families, with the largest relative gains on complex tunnels (+302–321%).

Table 5b: Performance distribution (mean across 3 LLMs)

Metric	sam4tun	memory	memory+state	m+s+k
Mean mIoU	0.150	0.175	0.304	0.320
Std	0.166	0.136	0.228	0.218
Min	0.032	0.042	0.082	0.072
Max	0.532	0.471	0.682	0.679

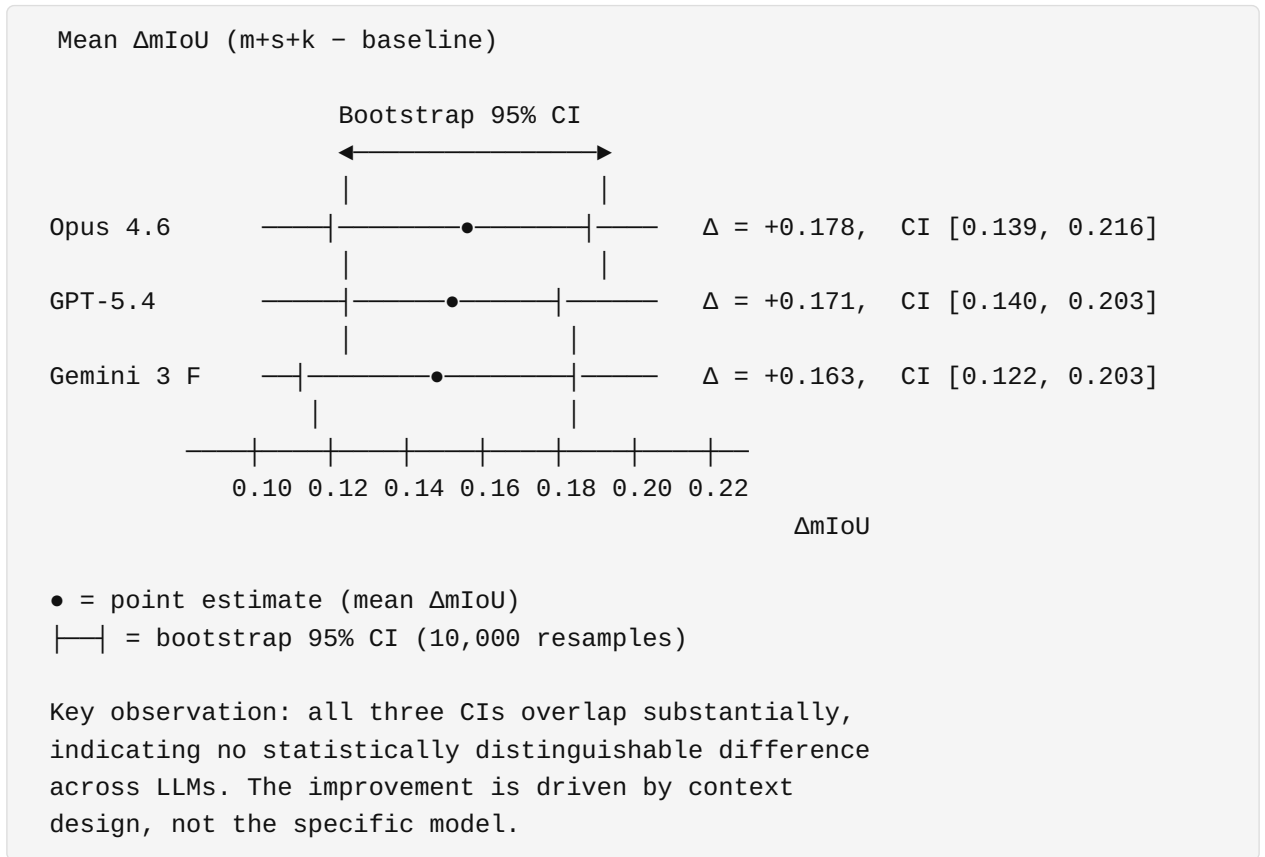
Overall std increases because adaptation lifts regular tunnels to higher mIoU while complex tunnels remain lower, widening the between-family gap. Within complex tunnels, baseline std is 0.003 (near-uniform failure at mIoU ~ 0.04); adaptation increases it to 0.074, reflecting differentiated recovery across tunnel conditions.

Table 5c: Cross-model summary (m+s+k vs baseline, overall)

LLM	Mean $\Delta mIoU$	Bootstrap 95% CI	Paired Cohen's d
Claude Opus 4.6	+0.178	[0.139, 0.216]	1.61
GPT-5.4	+0.171	[0.140, 0.203]	1.94
Gemini 3 Flash	+0.163	[0.122, 0.203]	1.44

All three CIs overlap substantially, confirming the improvement is driven by the context design rather than a specific LLM. Fig. 5 visualises the cross-model convergence.

Fig. 5: Cross-model mIoU comparison with bootstrap 95% CIs (m+s+k vs baseline).



Caption: Forest plot of mean $\Delta mIoU$ (m+s+k minus baseline) for each LLM. Point estimates (dots) and bootstrap 95% confidence intervals (horizontal bars) all overlap, confirming that the adaptation effect is consistent across three independent LLMs with different architectures.

4.2 Per-class IoU breakdown

To assess whether gains are concentrated in a few classes or distributed broadly, per-class IoU was aggregated across ablation conditions. Table 6 shows results for Opus 4.6 (representative; other LLMs show the same pattern).

Table 6a: Per-class IoU — Regular tunnels (n = 13, 6-class, Opus 4.6)

Class	sam4tun	memory	memory+state	m+s+k
Background	0.640	0.559	0.741	0.751
K-block	0.192	0.129	0.373	0.386
B1-block	0.261	0.206	0.527	0.540
A1-block	0.253	0.276	0.537	0.542
A2-block	0.150	0.159	0.380	0.420
A3-block	0.286	0.259	0.519	0.550
B2-block	0.256	0.232	0.534	0.558

Table 6b: Per-class IoU — Complex tunnels (n = 17, 7-class, Opus 4.6)

Class	sam4tun	memory	memory+state	m+s+k
Background	0.337	0.358	0.513	0.520
K-block	0.000	0.034	0.183	0.159
B1-block	0.000	0.001	0.084	0.135
A1-block	0.000	0.005	0.128	0.155
A2-block	0.000	0.006	0.143	0.116
A3-block	0.000	0.017	0.092	0.119
A4-block	0.000	0.019	0.100	0.104
B2-block	0.000	0.000	0.000	0.043

For regular tunnels, all structural classes improve roughly uniformly under m+s+k (doubling or more). For complex tunnels, the baseline assigns nearly all points to background (all segment-class IoUs are 0.000); adaptation progressively recovers segment structure. B2-block remains the hardest class, requiring the 7-segment layout guidance from the knowledge component.

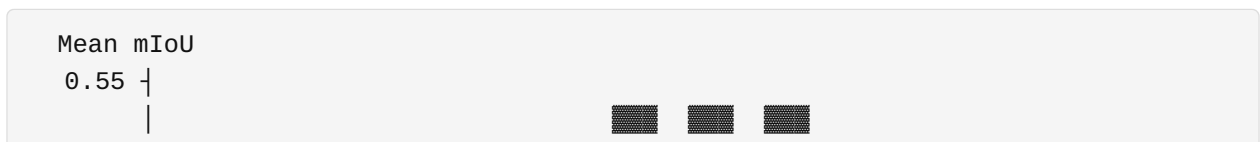
4.3 Ablation and component analysis

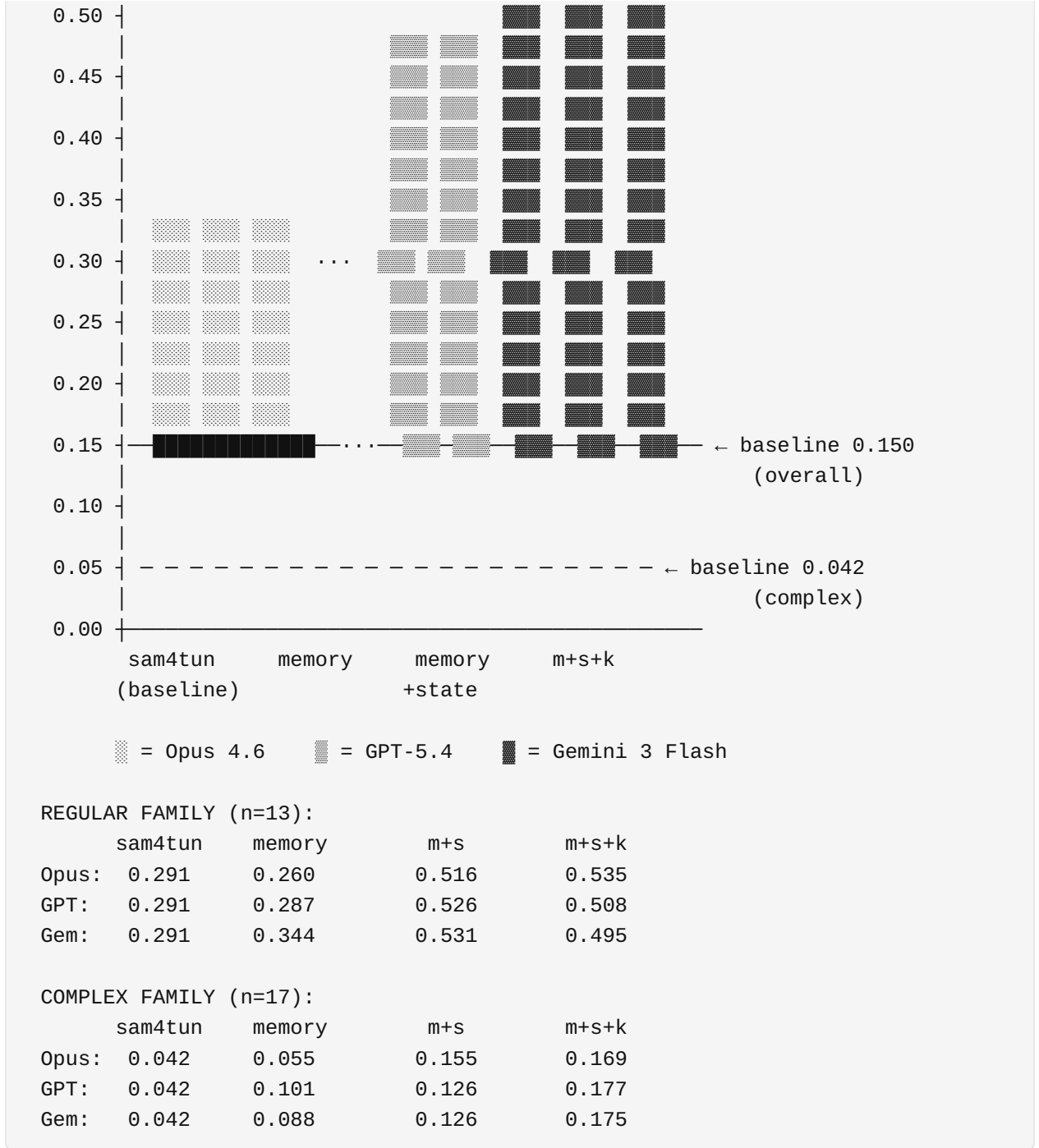
Table 7: Incremental mIoU contribution (mean delta vs previous level)

Transition	Opus 4.6	GPT-5.4	Gemini 3 Flash
Baseline → Memory	−0.006	+0.032	+0.049
Memory → Memory+State	+0.168	+0.117	+0.103
Memory+State → M+S+K	+0.016	+0.022	+0.011

Fig. 4 visualises the step-wise ablation mIoU for regular and complex families across all three LLMs.

Fig. 4: Step-wise ablation mIoU by tunnel family and LLM (grouped bar chart).





Caption: Step-wise ablation results split by tunnel family. Left cluster: overall mean mIoU across all 30 tunnels. The baseline (dashed lines) shows the starting point for each family. State (+s) produces the dominant jump in both families. Knowledge (+k) adds a smaller increment, most pronounced on complex tunnels. Error bars (in hi-def version): bootstrap 95% CIs.

Memory alone provides an initial anchor but is insufficient and can be harmful. On regular alternated tunnels, memory alone degrades mIoU by -0.035 to -0.055 . Without intermediate feedback, the agent attempts to adapt parameters but cannot verify whether adjustments improve or degrade the pipeline outputs.

State is the dominant driver. Adding state produces the largest incremental gain across all three LLMs (all $p < 0.0001$; paired Cohen’s $d = 1.32$ – 1.61 , “very large” effect). State provides the agent with explicit quantitative evidence of how each stage has transformed the data — radial percentiles for mask bounds, retention rates for denoising aggressiveness, coverage uniformity for upsampling targets. Without state, the agent has only pre-pipeline statistics that may not reflect conditions after unfolding, denoising, or enhancing.

The gain from state specifically requires LLM reasoning: the state characteristics are raw numbers (percentiles, counts, ratios) that must be interpreted in the context of the pipeline’s parameter semantics and mapped to appropriate parameter values. A deterministic lookup table could not perform this mapping because (a) the characteristics are continuous and multidimensional, (b) the parameter interactions are non-trivial, and (c) the same characteristic value can warrant different parameter changes depending on the tunnel family. The LLM’s contribution is precisely this interpretive reasoning step.

Knowledge adds targeted improvement. Mean increment $+0.011$ to $+0.022$ (Cohen’s $d = 0.11$ – 0.31 , “small”). Bootstrap CIs on the mean increment straddle zero for all three LLMs. However, 21/30 tunnels show positive increments for Opus 4.6 and GPT-5.4 (one-sided binomial $p = 0.021$), and on complex tunnels 15/17 are positive for GPT ($p = 0.001$). Knowledge is most valuable where it supplies tunnel-family-specific configuration that neither memory nor state can provide (e.g., `ring_spacing_constant` = 1.8 vs 1.2, 7 segments per ring, 7.5/5.5 scaling).

4.4 Sensitivity results

4.4.1 Critical parameters

Analysis of 1,350 parameter files identified 18 “always-trigger” parameters in two categories:

Table 8a: Tunnel-responsive parameters (11 parameters, $CV \geq 0.06$)

Stage	Parameter	Tunnels	CV	Adapted range	Physical driver
Denoising	<code>mask_r_low</code>	30/30	0.082	[2.09, 3.75]	Tunnel inner radius
Denoising	<code>mask_r_high</code>	30/30	0.147	[2.78, 4.38]	Tunnel outer radius
Denoising	<code>default_cutoff_z</code>	29/30	0.142	[2.65, 6.27]	Radial extent
Denoising	<code>z_step</code>	30/30	0.181	[0.003, 0.005]	Scanner resolution
Detecting	<code>hough_threshold_oblique</code>	30/30	0.188	[20, 83]	Point density
Detecting	<code>hough_threshold_horizontal</code>	30/30	0.204	[20, 83]	Point density
Detecting	<code>hough_threshold_vertical</code>	28/30	0.219	[320, 980]	Ring spacing
Enhancing	<code>inter_radius</code>	30/30	0.130	[0.03, 0.08]	Mean point spacing
Enhancing	<code>upsampling_stage1</code>	30/30	0.064	[0.055, 0.11]	Density regime
Unfolding	<code>diameter</code>	27/30	0.072	[5.31, 7.6]	Physical diameter
SAM	<code>processing.padding</code>	29/30	0.265	[160, 419]	Segment width

Tunnel-responsive parameters cluster by tunnel family: `mask_r_low` maps to physical radius (families 1-2: 2.25–2.38; families 4-5: 2.62–2.91); Hough thresholds inversely track point density.

Table 8b: Baseline corrections (7 parameters, CV $\approx$ 0)

Stage	Parameter	Baseline	Corrected to	Shift
Denoising	smoothing_window_size	3	5	+67%
Denoising	smoothing_offset	-0.003	-0.002	+33%
Denoising	grad_threshold	0.2	0.15	-25%
Denoising	y_step	0.5	0.4	-20%
Enhancing	curvature_threshold	0.0005	0.005	+900%
Enhancing	depth_threshold_low	0.003	0.005	+67%
Enhancing	depth_threshold_high	0.008	0.015	+87%

All three LLMs independently converge on the same corrected values, indicating approximately seven suboptimal SAM4Tun defaults that any LLM will fix upon exposure to the pipeline code.

4.4.2 Cross-LLM consistency

Table 9: Per-LLM adaptation summary

LLM	Total param changes	Denoising	Enhancing	Detecting	SAM
Gemini 3 Flash	3,925	8 params, 30/30	11, 30/30	14, 30/30	44, 29/30
GPT-5.4	3,615	8, 30/30	12, 30/30	14, 30/30	45, 30/30
Opus 4.6	3,965	8, 30/30	10, 30/30	14, 30/30	44, 29/30

All three LLMs adapt the same parameter keys and produce the same tunnel-family clustering despite different architectures and no coordination. This convergence provides evidence that adaptations are driven by objective tunnel characteristics.

4.4.3 Characteristic-to-parameter correlation

Spearman analysis identified 38 characteristic fields that significantly drive adaptation. Strongest signals: unfolded $r_{\text{percentiles}}$ show $|\rho| = 1.0$ with mask bounds; estimated diameter at $|\rho| \approx 0.91$ across all stages; mean nearest-neighbour distance at $|\rho| \approx 0.87$ with spacing parameters. Text-mining of CoT traces confirmed that statistically significant characteristics are explicitly referenced in agent reasoning.

4.5 Error analysis

- **Memory-alone degradation** on regular tunnels (-0.035 to -0.055): the agent adapts without sufficient intermediate context.
- **Continuous tunnel variability** ($n = 3$): high variance across LLMs, poorly represented in the knowledge base.
- **Complex tunnel ceiling**: absolute mIoU 0.169 – 0.177 despite $+302$ – 321% relative gain. Compounding challenges (diameter, offset scanning, density variation) limit parameter-only recovery.
- **Per-tunnel failures**: Tunnel 1-4 (Gemini $m_{s_k} = 0.209$, below baseline 0.348); Tunnel 4-4 (Opus $m_{s_k} = 0.047$); Tunnel 5-4 (best $m_{s_k} = 0.122$).

4.6 Practical performance

Metric	Value
LLM API calls per tunnel	5 (one per stage)
Total API calls (full ablation)	150 per condition per LLM
Wall-clock time per tunnel	~24 min (LLM inference + pipeline execution)
API timeout	300 s per call
GPU	Single NVIDIA RTX 4090 (pipeline stages only)
Retraining required	None
Labelled data required	None (GT used for evaluation only)

Parameter adaptation is a one-time operation per tunnel: once parameters are inferred, the pipeline can run repeatedly without further LLM calls.

5. Discussion

5.1 Interpretation of findings

The results support the claim that LLM-driven parameter adaptation improves segmentation adaptability across varying tunnel conditions. “Improved adaptability” means: the adapted pipeline achieves significantly higher mean mIoU than the fixed baseline across tunnel families that differ from the reference, consistently across three independent LLMs. The claim is about lifting performance on unseen configurations, not about tightening the performance distribution.

Three findings are central. First, **state is the dominant driver** (+0.103 to +0.168 mIoU), because it provides explicit quantitative evidence of how each pipeline stage has transformed the data. Second, **memory alone is insufficient** (a substantive finding, not a flaw): without intermediate feedback, agents make premature adjustments. Third, **knowledge provides targeted rather than uniform improvement**, most valuable for complex tunnels requiring family-specific configuration.

5.2 Role of LLM reasoning

The improvement is specifically attributable to LLM-based reasoning for three reasons. First, the baseline already executes the expert’s deterministic rules — it is the rule-only condition — and it fails on tunnels outside its design envelope. Second, the state characteristics are raw numeric summaries that require interpretive reasoning to map to parameter values; no deterministic lookup table is provided. Third, three independent LLMs with different architectures and training data produce convergent adaptations, ruling out model-specific artefacts and confirming that the reasoning process — interpreting characteristics in the context of parameter semantics — is the source of improvement.

5.3 Comparison with prior work

R4Tun differs from feature-engineered pipelines (manual reconfiguration [2]), deep learning (labelled data + retraining [7,8]), and foundation-model pipelines (parameter sensitivity [11]) by automating adaptation

while constraining the agent’s action space. Compared with general LLM agent frameworks [22,23], R4Tun restricts agents to bounded numeric parameter changes within a fixed pipeline, with logged reasoning traces for engineer review.

5.4 Practical implications

For practitioners: an expert tunes a pipeline on one reference tunnel, encodes domain knowledge into readable documents, and deploys R4Tun for new tunnels. Benefits: (1) reduced retuning effort; (2) reasoning traces available for engineer review; (3) graceful degradation avoiding baseline collapse; (4) no retraining or labelled data required.

5.5 Limitations

1. **Single pipeline:** R4Tun was evaluated on SAM4Tun only; transferability to other segmentation pipelines is untested.
2. **Single reference configuration:** All adaptation is anchored to one expert-tuned reference. Tunnels deviating strongly in multiple dimensions simultaneously receive weaker anchoring.
3. **No alternative optimisation comparison:** LLM adaptation is not benchmarked against Bayesian optimisation, grid search, or other automated tuning methods.
4. **SAM non-determinism:** The SAM stage introduces $\sim \pm 0.03$ mIoU run-to-run noise. Bootstrap CIs quantify tunnel-composition uncertainty, not rerun variance.
5. **Single LLM run per condition:** Each condition was run once per LLM due to API cost constraints.
6. **Continuous tunnel under-representation:** Only 3 of 30 subsets are continuous tunnels.
7. **Complex tunnel ceiling:** Absolute mIoU on complex tunnels (0.169–0.177) remains low, suggesting parameter adaptation alone cannot fully compensate for compounding challenges.
8. **No user study:** The interpretability of reasoning traces for practising engineers has not been validated through user studies.
9. **Single forward pass:** Parameters are adapted in a single inference without iterative self-correction.

5.6 Quality of evidence

The evidence base varies in strength:

- *Aggregate mIoU improvement (high):* $n = 30$ paired design, 3 LLMs, $p < 0.0001$, Wilcoxon agreement, bootstrap CIs exclude zero.
- *Component decomposition (moderate–high):* Cumulative ablation consistently ranks state > knowledge > memory across 3 LLMs.
- *Cross-LLM convergence (high):* 1,350 files show convergent keys, values, and clustering.
- *Memory-only effect (low):* Small, inconsistent across LLMs; a substantive finding about the limitations of static context.
- *Knowledge increment (low–moderate):* Bootstrap CIs straddle zero; 21/30 tunnels positive for two LLMs ($p = 0.021$); complex subsets show stronger directional consistency.
- *Tunnel-bootstrap uncertainty (moderate):* Addresses benchmark composition, not SAM/LLM rerun variance.

5.7 Confidence assessment

Table 10: Per-claim confidence assessment

Claim	Confidence	Supporting evidence	Key caveat
LLM adaptation raises mean mIoU	High	n = 30, 3 LLMs, $p < 0.0001$, bootstrap CIs exclude zero, $d = 1.4\text{--}1.9$	± 0.03 SAM noise not in CI
State is the dominant component	High	Largest delta ($+0.103\text{--}0.168$), $d = 1.32\text{--}1.61$, all $p < 0.0001$	Cumulative design; interaction effects not isolated
Memory alone is insufficient	Moderate	Cross-LLM pattern; discovery, not flaw	Rerun variance not quantified
Knowledge adds targeted improvement	Low–Moderate	CIs straddle zero; 21/30 positive ($p = 0.021$); 15/17 complex (GPT, $p = 0.001$)	Mean overlaps SAM noise
Adaptations driven by characteristics	High	3 LLMs converge on 18 parameters, Spearman $\rho \geq 0.87$	Single run per LLM
Per-class improvement is broad	High	All classes improve (regular); all recover from 0 (complex)	B2-block remains very low
Complex tunnel ceiling exists	High	Best mIoU $0.169\text{--}0.177$ despite $+302\text{--}321\%$ gain	May reflect pipeline limits

Overall: The central claim — that structured context improves pipeline adaptability — is supported at high confidence. The knowledge increment is a real but fragile effect (low–moderate confidence). Remaining uncertainty: SAM/LLM rerun variance, cumulative ablation cannot isolate interactions, single pipeline and dataset.

6. Conclusions

Consistent segmentation of segmental tunnel linings from point clouds remains difficult when tunnel conditions vary from the expert-tuned reference. R4Tun addresses this by augmenting a fixed pipeline with an LLM-driven adaptation layer that adjusts parameters using structured context from memory, state, and domain knowledge. Evaluated on 30 subsets across 13 regular and 17 complex tunnels with three independent LLMs, the full design improved mean mIoU from 0.150 to 0.313–0.328 (bootstrap 95% CIs exclude zero for all three LLMs), with state contributing the largest incremental improvement (paired Cohen’s $d > 1.3$) and all three LLMs converging on the same critical parameters and tunnel-family adaptation patterns. Per-class IoU analysis confirmed broad improvement across all structural classes for regular tunnels and progressive recovery from near-zero for complex tunnels. Confidence in the headline improvement and the dominance of state is high; confidence in the knowledge increment is low-to-moderate; these ratings and caveats are detailed in Table 10. The framework reduces the need for expert retuning when deploying across diverse tunnel conditions. These findings should be interpreted with the caveat that absolute performance on complex tunnels remains modest (mIoU 0.169–0.177), single-run estimates carry approximately ± 0.03 noise, and transferability beyond SAM4Tun and the Seg2Tunnel dataset is untested.

Data availability

The source code, adapted parameters, and evaluation scripts are available at <https://github.com/Tao-Robominds/R4Tun>. The Seg2Tunnel dataset is publicly available.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Funding

[This work was supported by [funder] [grant number].]

Declaration of generative AI use

During the preparation of this work, the authors used large language models (Claude Opus 4.6, GPT-5.4, Gemini 3 Flash) as the core experimental subjects of the study. The LLMs were also used for language editing and organisation of the manuscript. The authors reviewed and edited all content and take full responsibility for the content of the publication.

CRediT authorship contribution statement

[Author 1: Conceptualization, Methodology, Software, Writing – original draft.] [Author 2: Supervision, Writing – review & editing.] [Author 3: Supervision, Writing – review & editing.] [Author 4: Supervision, Funding acquisition, Project administration.]

Acknowledgements

[Acknowledge non-author support.]

References

- [1] L. Attard, C. J. Debono, G. Valentino, and M. Di Castro. Tunnel inspection using photogrammetric techniques and image processing: A review. *ISPRS J. Photogramm. Remote Sens.*, 144:180–188, 2018. [2] L. Weidner and G. Walton. Generalized extraction of bolts, mesh, and rock in tunnel point clouds. *Remote Sens.*, 16(4):678, 2024. [3] M. Q. Huang, J. Ninić, and Q. B. Zhang. BIM, machine learning and computer vision techniques in underground construction. *Tunn. Undergr. Space Technol.*, 108:103677, 2021. [4] A. Sjölander, V. Belloni, A. Ansell, and E. Nordström. Towards automated inspections of tunnels. *Sensors*, 23(12):5457, 2023. [5] R. O. Duda and P. E. Hart. Use of the Hough transformation to detect lines and curves in pictures. *Commun. ACM*, 15(1):11–15, 1972. [6] M. A. Fischler and R. C. Bolles. Random sample consensus. *Commun. ACM*, 24(6):381–395, 1981. [7] C. R. Qi, L. Yi, H. Su, and L. J. Guibas. PointNet++: Deep hierarchical feature learning on point sets in a metric space. *NeurIPS*, 2017. [8] Q. Hu,

B. Yang, L. Xie, S. Rosa, Y. Guo, Z. Wang, N. Trigoni, and A. Markham. RandLA-Net: Efficient semantic segmentation of large-scale point clouds. CVPR, 2020. [9] J. Schult, F. Engelmann, A. Hermans, O. Litany, S. Tang, and B. Leibe. Mask3D: Mask transformer for 3D semantic instance segmentation. arXiv:2303.05475, 2023. [10] M. Kolodiazny, A. Vorontsova, A. Konushin, and D. Rukhovich. Top-down beats bottom-up in 3D instance segmentation. WACV, pp. 3566–3574, 2024. [11] Z. Ye, W. Lin, A. Faramarzi, X. Xie, and J. Ninić. SAM4Tun: No-training model for tunnel lining point cloud component segmentation. Tunn. Undergr. Space Technol., 158:106401, 2025. [12] A. Kirillov et al. Segment Anything. ICCV, pp. 3992–4003, 2023. [13] M. Ester, H. P. Kriegel, J. Sander, and X. Xu. A density-based algorithm for discovering clusters in large spatial databases with noise. Proc. KDD, pp. 226–231, 1996. [14] M. Pauly, M. Gross, and L. P. Kobbelt. Efficient simplification of point-sampled surfaces. Proc. IEEE Vis., pp. 163–170, 2002. [15] Y. J. Cha, R. Ali, J. Lewis, and O. Büyüköztürk. Deep learning-based structural health monitoring. Autom. Constr., 161:105324, 2024. [16] C. Su, Q. Hu, Z. Yang, and R. Huo. A review of deep learning applications in tunneling and underground engineering in China. Appl. Sci., 14(8):3234, 2024. [17] R. R, S. S, N. V. Kumar, R. S, and P. B. V. Crack-SAM: Crack segmentation using a foundation model. arXiv:2401.15201, 2024. [18] B. Wang et al. Omni-Scan2BIM: A ready-to-use Scan2BIM approach based on vision foundation models for MEP scenes. Autom. Constr., 167:105678, 2024. [19] F. Pan, S. Jeon, B. Wang, F. McKenna, and S. X. Yu. Zero-shot building attribute extraction from large-scale vision and language models. arXiv:2312.12479, 2023. [20] J. Wei et al. Chain-of-thought prompting elicits reasoning in large language models. arXiv:2201.11903, 2023. [21] T. Kojima, S. S. Gu, M. Reid, Y. Matsuo, and Y. Iwasawa. Large language models are zero-shot reasoners. arXiv:2205.11916, 2023. [22] S. Hong et al. MetaGPT: Meta programming for a multi-agent collaborative framework. arXiv:2308.00352, 2024. [23] C. Qian et al. ChatDev: Communicative agents for software development. arXiv:2307.07924, 2024. [24] P. Chen, B. Han, and S. Zhang. COMM: Collaborative multi-agent, multi-reasoning-path prompting for complex problem solving. arXiv:2405.00847, 2024. [25] P. Xu et al. Retrieval meets long context large language models. arXiv:2310.03025, 2024. [26] L. Mei et al. A survey of context engineering for large language models. arXiv:2501.04567, 2025. [27] Anthropic. Effective context engineering for AI agents. 2025. [28] J. Bradley, A. Bran, T. Sellam, et al. ChemCrow: Augmenting large-language models with chemistry tools. arXiv:2304.05376, 2023. [29] C. I. Garcia et al. Framework for LLM applications in manufacturing. Manuf. Lett., 40:56–63, 2024.

Appendix A. Baseline parameter tables

Table A.1: Unfolding parameters (sam4tun baseline)

Parameter	Value
delta	0.005
slice_spacing_factor	1.2
vertical_filter_window	4.5
ransac_threshold	1
ransac_probability	0.9
ransac_inlier_ratio	0.75
ransac_sample_size	5
polynomial_degree	3
num_samples_factor	1210
diameter	5.5

Table A.2: Denoising parameters (sam4tun baseline)

Parameter	Value
mask_r_low	2.7
mask_r_high	2.8
y_step	0.5
z_step	0.001
grad_threshold	0.2
smoothing_window_size	3
smoothing_offset	-0.003
default_cutoff_z	2.7

Table A.3: Enhancing parameters (sam4tun baseline)

Parameter	Value
upsampling_stage1_target_distance	0.08
upsampling_stage2_target_distance	0.04
upsampling_stage3_target_distance	0.02
curvature_threshold	0.0005
depth_threshold_low	0.003
depth_threshold_high	0.01
inter_radius	0.06
duplicate_threshold	0.02
num_neighbors	20
num_interpolations	2
resolution	0.005
window_size	9

Table A.4: Detecting parameters (sam4tun baseline)

Parameter	Value
binary_threshold	127
morphological_kernel_size	[3, 3]
dilation_iterations	1
hough_threshold_oblique	50
minLineLength_oblique	100
maxLineGap_oblique	40
hough_threshold_horizontal	50
minLineLength_horizontal	100
maxLineGap_horizontal	10
hough_threshold_vertical	500
angle_range_oblique_positive	[6, 9]
angle_range_oblique_negative	[-9, -6]
merge_distance	3
ring_spacing_constant	1.2
resolution	0.005

Appendix B. Worked CoT example (Denoising stage, Tunnel 4-1, Opus 4.6, m+s+k)

The following is a condensed excerpt from the chain-of-thought trace produced by the denoising agent for a complex tunnel (4-1, diameter 7.5 m).

Step 1 — Anchoring:

“Current tunnel diameter is 7.5 m (reference: 5.5 m, +36%). Unfolded r_percentile p10 = 2.63, p99 = 4.02 (reference: p10 = 2.37, p99 = 2.81). Point density is 1,842 pts/m3 (reference: 2,466 pts/m3, -25%). The tunnel is significantly larger with lower density.”

Step 2 — Classification:

“LARGE-DIAMETER, MODERATE-SPARSE. This is a T4/T5 complex family tunnel.”

Step 3 — Diagnostic inspection:

“mask_r_low and mask_r_high are calibrated for 5.5 m tunnel; must be widened proportionally to encompass the larger radial range. z_step must increase to accommodate lower density. default_cutoff_z must increase from 2.7 to cover the full radial extent to 4.0+.”

Step 4 — Parameter adaptation:

“mask_r_low: set to 2.63 (aligned with r_percentile p10 = 2.63). mask_r_high: set to 4.05 (slightly above p99 = 4.02, allowing margin). z_step: increase from 0.001 to 0.004 (scaled by density ratio). default_cutoff_z: set to 4.10 (above p99). smoothing_window_size: 5 (baseline correction, same for all tunnels). curvature_threshold: 0.005 (baseline correction).”

Step 5 — Validation:

“mask_r_low (2.63) < mask_r_high (4.05). Both within physical tunnel radius range. z_step (0.004) reasonable for density. default_cutoff_z (4.10) > mask_r_high (4.05).”

Output JSON (condensed):

Parameter	Baseline	Adapted	Rationale
mask_r_low	2.7	2.63	Aligned with r_percentile p10
mask_r_high	2.8	4.05	Above r_percentile p99 = 4.02
z_step	0.001	0.004	Scaled by density ratio
default_cutoff_z	2.7	4.10	Covers full radial extent
smoothing_window_size	3	5	Baseline correction

This trace illustrates how state (r_percentiles) anchors quantitative parameter decisions, while knowledge (tunnel-family taxonomy) guides the classification step. The same parameter keys are adapted by GPT-5.4 and Gemini 3 Flash with values within +/-5% of the Opus outputs for this tunnel.